

By-Hand Replication Guide: Integrating AR5-L6 & LinkerHand O6

An exhaustive, click-by-click, line-by-line manual to reproduce the entire ROS 2 / Isaac Sim bridge from scratch.

This document is designed so that a brand new researcher can blindly follow these exact steps and reproduce 100% of our progress from today. It covers exactly what to click in Isaac Sim and exactly what code to paste in ROS 2.

Phase 1: Isaac Sim Physics Stabilization (GUI Steps)

If you import the raw URDF into Isaac Sim and press Play, the robot will explode. Follow these exact GUI steps to stabilize the PhysX solver.

Step 1.1: Define the Articulation Root

PhysX must know where the robot's hierarchical tree begins.

1. Open Isaac Sim. In the top-left **Stage** tree, click the highest-level parent folder of the robot (e.g., `ar5_o6_left_combined`).
2. Look at the **Property** panel on the right side of the screen.
3. Scroll to the top of the Property panel and click the **+ Add** button.
4. Select **Physics** → **Articulation Root**.
5. *Crucial:* Expand the entire robot tree and ensure no other link has an Articulation Root. If one does, right-click it in the Property panel and delete it.

Step 1.2: Add Virtual Inertia (Armature) to Prevent Explosions

The 1:1000 mass ratio between the arm and fingers destroys the math solver. We must add Armature.

1. In the **Stage** tree, hold **Ctrl** and click to select **every single revolute joint** in the arm and the hand simultaneously.
2. In the **Property** panel, click **+ Add** → **Physics** → **Joint**. (This exposes hidden PhysX parameters).
3. Scroll down to the newly added Physics parameters and find the **Armature** field.
4. Type `0.05` and press **Enter**.

Step 1.3: Add Position Drive (Muscle Tension)

Without motor stiffness, the heavy arm will instantly collapse under gravity.

1. With all joints still selected, click **+ Add** → **Physics** → **Angular Drive**.
2. Scroll down to the Drive section in the Property panel.
3. Set **Stiffness** to `400.0`.
4. Set **Damping** to `40.0`.

Step 1.4: Anchor the Base Link

If you hit play now, the robot will infinitely fall through the floor.

1. Select the very first rigid body link in the Stage tree (e.g., `AR5_5_07L_W4C4A2_base`).

2. In the Property panel, under the Rigid Body section, check the box for **Disable Gravity** (or create a Fixed Joint anchoring it to the World).

Phase 2: Fixing MoveIt Configuration Crashes (Code Edits)

The MoveIt Setup Assistant generates files with fatal flaws. Navigate to your workspace (/media/omen/.../ahmed_moveit/config/) and make these exact edits.

Step 2.1: Fix the C++ Integer Crash

THE CRASH

ROS 2 instantly aborts on launch: parameter 'joint_limits.max_velocity' has invalid type: expected double, got integer.

1. Open config/joint_limits.yaml.
2. Find every value that is a whole number (e.g., max_velocity: 1).
3. Change them to floating-point decimals (e.g., max_velocity: 1.0). *Do this for every single joint.*

Step 2.2: Fix Time Parameterization (Acceleration Limits)

RViz will plan a path, but clicking "Execute" will fail with a Time Parameterization error because the trajectory generator (TOTG) doesn't know how fast the motors can accelerate.

1. In config/joint_limits.yaml, under every joint, add these two lines exactly:

```
has_acceleration_limits: true
max_acceleration: 5.0
```

Step 2.3: Fix the Hand Controller Crash (GripperCommand)

MoveIt crashes trying to load a GripperCommand for a 6-DoF hand.

1. Open config/moveit_controllers.yaml.
2. Locate the hand_controller section.
3. Change action_ns: gripper_cmd to action_ns: follow_joint_trajectory.
4. Change the type to FollowJointTrajectory. The final block should look like this:

```
- name: hand_controller
  action_ns: follow_joint_trajectory
  default: true
  type: FollowJointTrajectory
  joints:
    - lh_thumb_cmc_yaw
    - lh_thumb_cmc_pitch
    - lh_index_mcp_pitch
    - lh_middle_mcp_pitch
    - lh_ring_mcp_pitch
    - lh_pinky_mcp_pitch
```

Phase 3: Dexterous Hand Specifics (Code Edits)

The LinkerHand O6 has extremely dense collision meshes and passive tendons (mimic joints). These completely break MoveIt.

Step 3.1: Delete the Passive Mimic Joint from Planning

MoveIt will instantly abort planning if a passive slaved joint is included in an active planning group.

1. Open `config/ar5_o6_left.srdf`.
2. Search for `lh_thumb_ip`.
3. Delete the line `<joint name="lh_thumb_ip"/>` from the `<group name="hand">` tag.
4. Delete `lh_thumb_ip` from *every* `<group_state>` definition (like the "thumps" pose).

Step 3.2: Disable Dense Hand Self-Collisions

When closing the fist, the finger bounding boxes intersect, triggering MoveIt's collision safety abort.

1. At the bottom of `config/ar5_o6_left.srdf`, paste `<disable_collisions>` tags connecting every finger to every other finger, and the palm. (*We used a python script to generate a 78-line matrix for this*).

Step 3.3: Sync the Spinal Cord (`ros2_controllers.yaml`)

THE EXECUTION SYNC ABORT

If you don't do this, RViz will say "Starting trajectory execution..." and then immediately abort with "Action client not connected".

1. Open `config/ros2_controllers.yaml`.
2. Ensure the `hand_controller` is defined exactly like the arm controller:

```
hand_controller:
  ros__parameters:
    joints:
      - lh_thumb_cmc_yaw
      - lh_thumb_cmc_pitch
      - lh_index_mcp_pitch
      - lh_middle_mcp_pitch
      - lh_ring_mcp_pitch
      - lh_pinky_mcp_pitch
    interface_name: position
```

Step 3.4: Fix the Stiff Fingers Visual Bug

Fake hardware ignores passive mimic joints. We must mathematically force it to calculate the tip joints.

1. Open `config/ar5_o6_left.ros2_control.xacro`.
2. Scroll to the bottom, right before the `</ros2_control>` tag, and explicitly paste these mathematically injected mimic definitions:

```

<joint name="lh_thumb_ip">
  <param name="mimic">lh_thumb_cmc_pitch</param>
  <param name="multiplier">2.29</param>
  <state_interface name="position" />
</joint>
<joint name="lh_index_dip">
  <param name="mimic">lh_index_mcp_pitch</param>
  <param name="multiplier">0.89</param>
  <state_interface name="position" />
</joint>
<!-- Repeat for middle, ring, and pinky -->

```

Step 3.5: The Colcon Build Trap (CRITICAL)

ROS 2 reads files from the compiled `install/` directory, NOT the source folder you just edited!

1. Open a terminal in `/media/omen/88D2C6C4D2C6B5AA/TianjinUniversity/ros2_ws`.
2. Ensure Conda is deactivated from your path: `export PATH=$(echo $PATH | sed -e 's|/home/omen/miniforge3/bin:||')`
3. Run: `colcon build --packages-select ahmed_moveit`
4. Run: `source install/setup.bash`

Phase 4: Wiring the Isaac Sim Action Graph

The robot still won't move until the ROS 2 data is cleanly streamed into PhysX.

Step 4.1: Create the Nodes

1. In Isaac Sim, go to **Window** → **Visual Scripting** → **Action Graph**.
2. Click **New Action Graph**.
3. Right-click in the empty space and search/add these three nodes:
 - On Playback Tick
 - ROS2 Subscribe Joint State
 - Articulation Controller

Step 4.2: Configure the Topic and Target

1. Click the **ROS2 Subscribe Joint State** node. In the Property panel, change `topicName` to `/joint_states`. **You MUST press Enter after typing, or Isaac Sim will secretly revert it.**
2. Click the **Articulation Controller** node. In the Property panel, look for **Target**. Click **Add Target**, then select the robot's Articulation Root from the Stage tree (`ar5_o6_left_combined`).

Step 4.3: Prevent the Race Condition

If you wire the execution pins in parallel, the Controller fires before the Subscriber finishes downloading data.

1. Click and drag the white **Exec Out** pin of On Playback Tick to the white **Exec In** pin of ROS2 Subscribe Joint State.
2. Click and drag the white **Exec Out** pin of ROS2 Subscribe Joint State to the white **Exec In** pin of Articulation Controller.

COMPLETION

If you have followed every step above perfectly, you can now run `ros2 launch ahmed_moveit demo.launch.py`, click "Plan & Execute" in RViz, and watch the fully-simulated physical robot in Isaac Sim mirror the movement flawlessly, with perfectly curled mimic fingers. You are ready for Milestone 3.